LangChain-M-H-Set2

FDE / Architect practice | Reasoning, alternatives, and design takeaways

01. Authorization before external reranking

Answer B: Apply enforced access filtering before external reranking, with backend filtering or a trusted pre-egress check.

The disclosure occurs when document text leaves the trusted boundary, not only when an answer reaches the user. Permission filtering must therefore occur before that egress. Use backend-enforced filtering where supported, or a trusted authorization stage before external processing. The integration's actual filtering behavior must be tested; a filter parameter alone is not proof of enforcement.

Why the alternatives fail

A. Removing a citation cannot undo text already transmitted to the reranker.

C. Instructions about output do not prevent the service from receiving the input.

D. A smaller unfiltered candidate pool still permits unauthorized disclosure.

Design takeaway: Place authorization checks before every boundary they are meant to protect.

Official references: [1], [2]

02. Retrieved instructions are untrusted data

Answer D: Treat retrieved text as data and restrict tool destinations and data access in execution policy.

Retrieved content should not acquire authority over the application. Separate it from trusted instructions and enforce tool capabilities, destinations, and data access outside the model. Prompt-based defenses can contribute, but an execution policy constrains what an injection can cause even if the model follows the hostile text.

Why the alternatives fail

A. Escaping punctuation does not neutralize instructions expressed in ordinary language.

B. Promoting source content to system authority strengthens the injection path.

C. A prompt alone is not an enforceable authorization or network boundary.

Design takeaway: Protect actions with deterministic policy, not solely with model obedience.

Official references: [2]

03. Identity is not a model-selected argument

Answer A: Derive caller identity from trusted runtime context and authorize the requested resource server-side.

A model-generated identifier is a request, not authentication. Use trusted request context for the caller's identity and enforce resource authorization in the tool or service. Runtime injection can keep identity out of the model-visible schema, but injection by itself is not the permission check. Delegated or cross-user access must be explicitly authorized.

Why the alternatives fail

B. A valid string type proves neither identity nor permission.

C. Conversation content is controlled by users and cannot establish access rights.

D. Removing descriptive text leaves the unauthorized execution path intact.

Design takeaway: Authenticate outside the model and authorize at the resource boundary.

Official references: [3]

04. Binding approval to the executed action

Answer C: Bind approval to the canonical tool name and arguments; reject or reapprove changed terms before execution.

Approval must refer to a specific action, not merely to a conversation. Store a canonical representation or digest of the approved tool and arguments, then compare the actual execution request at the trusted tool boundary. A material change invalidates that approval. Execution should also recheck current authorization and business preconditions.

Why the alternatives fail

A. New model context does not expand the human's authorization.

B. A session-wide flag authorizes changes the human never reviewed.

D. Post-action disclosure cannot satisfy a pre-execution approval requirement.

Design takeaway: Approve exact actions and verify that execution still matches them.

Official references: [4]

05. Output checks under a no-disclosure requirement

Answer A: Buffer the response before client delivery, validate or redact it, and release only the permitted result.

A complete-response check happens too late to prevent disclosure if tokens have already been sent. Under this requirement, buffer before client egress, then validate or redact and release. This sacrifices raw token immediacy for the stated guarantee. A different incremental design would need its own verified prevention properties; it cannot be assumed equivalent.

Why the alternatives fail

B. A later warning does not retract information already delivered.

C. Changing stored history does not erase what the client received.

D. Outputs can reproduce or infer sensitive information from tools and context, so input-only checks are insufficient.

Design takeaway: Align the enforcement point with when information actually leaves the system.

Official references: [2]

06. Choosing the fallback boundary

Answer B: (prompt_a | model_a | parser_a).with_fallbacks([prompt_b | model_b | parser_b])

Wrap the full primary transformation, including its parser, and provide a full alternate transformation with the same external input/output contract. The fallback then sees the original application input and can construct its own prompt. Configure the wrapper to handle the intended error classes. An error outside the protected runnable does not trigger its fallback.

Why the alternatives fail

A. The parser is outside the wrapper, and the alternate model receives the primary prompt.

C. The fallback receives the primary prompt value and omits its required alternate prompt and parser.

D. Parsing happens after the protected block, so a parser failure is outside fallback scope.

Design takeaway: Place fallbacks around the complete unit whose failure they must recover.

Official references: [5]

07. Capability-aware model routing

Answer D: Filter models by required capabilities first, then optimize cost among eligible models.

Request length is a weak proxy for the operation the model must perform. Build an eligibility check from tool use, structured-output requirements, context size, and any other hard constraints. Cost optimization operates inside that feasible set. Evaluate routed paths on representative cases rather than assuming a shared model interface implies identical capabilities.

Why the alternatives fail

A. A parser cannot supply an unexecuted tool operation or an unsupported provider feature.

B. Short requests can impose difficult or capability-specific tasks.

C. Dropping requirements changes the product contract instead of meeting it more cheaply.

Design takeaway: Optimize price only after satisfying the request's hard capability constraints.

Official references: [6], [7]

08. Critical-path latency in a retrieval chain

Answer A: 2,200 ms, because retrieval contributes max(800, 1400), followed by 300 and 500.

The chain waits for both independent retrieval outputs, so ideal parallel retrieval takes 1,400 ms. The dependent stages then add 300+500 ms, yielding 2,200 ms. The sequential baseline is 800+1,400+300+500=3,000 ms. This saves 800 ms of wall-clock latency, not the compute or API cost of the two retrieval operations.

Why the alternatives fail

B. That is the sequential baseline; independent retrieval branches overlap under the stated assumptions.

C. Both later stages depend on completed upstream work and cannot be omitted from the path.

D. The stated contract requires both results, so a first-finisher race would change behavior.

Design takeaway: Optimize the dependency critical path and distinguish latency savings from work savings.

Official references: [8]

09. Representing missing evidence in a schema

Answer C: Represent unknown separately, such as a nullable value plus evidence/status, and make consumers handle it explicitly.

Zero is a business value; unknown is an epistemic state. Conflating them creates misleading analytics even when JSON validation succeeds. Introduce an explicit missing-evidence representation and update downstream handling, tests, and schema versioning. A status such as not_disclosed can distinguish absence of disclosure from a sourced zero balance.

Why the alternatives fail

A. An undocumented sentinel creates a different silent semantic error.

B. Confidence wording does not change what the default value means to consumers.

D. A downstream zero default recreates the same conflation at another layer.

Design takeaway: Design output schemas to preserve uncertainty rather than manufacturing a business value.

Official references: [9]

10. Rank fusion across retrieval signals

Answer D: B ranks above A: approximately 0.03226 versus 0.03154.

For A, the score is 1/61+1/66, approximately 0.03154. For B, it is 1/62+1/62, approximately 0.03226. B's consistently high ranking outweighs A's first place in one list and weaker place in the other. Rank fusion avoids directly adding incompatible raw score scales, although weights and the rank constant remain design choices.

Why the alternatives fail

A. The supplied formula uses ranks, so raw similarity and lexical scores are unnecessary.

B. RRF sums contributions from all lists; one first-place result is not an unconditional winner.

C. Presence in the same number of lists does not erase differences in rank.

Design takeaway: Compute fusion from its actual rule; do not substitute an intuitive winner.

Official references: [10]

11. Decomposing a multi-fact retrieval task

Answer B: Retrieve the two event dates separately, retain their evidence, then compare the normalized dates.

The answer requires two facts, not simply a longer context about one event. Decompose into event-specific retrievals, preserve source support for each date, normalize date semantics, and perform the comparison. If either date remains unsupported or ambiguous, report that limitation. This makes evidence completeness explicit instead of assuming top-k similarity covers every needed fact.

Why the alternatives fail

A. More generation space does not supply a second event date absent from evidence.

C. One date cannot establish a temporal relationship between two events.

D. Further shortening can remove the distinction that retrieval already fails to cover.

Design takeaway: Map multi-fact questions to explicit evidence requirements before combining results.

Official references: [11], [1]

12. Resolving versioned evidence

Answer C: Select evidence applicable to the requested date and cite that version, rather than treating rank as temporal authority.

Relevance scoring does not determine which document was in force on a historical date. Use effective-date and version metadata to identify the applicable policy and preserve those details in the answer. If applicability metadata is missing or contradictory, flag the ambiguity rather than resolve it through similarity rank alone.

Why the alternatives fail

A. Similarity measures textual relevance, not temporal validity.

B. A later effective policy does not automatically change what applied before its effective date.

D. Combining conflicting rules can produce a policy that never existed.

Design takeaway: Use applicability metadata to resolve conflicts that similarity ranking cannot settle.

Official references: [12]

13. Removing stale indexed content

Answer A: Track source/revision membership and remove superseded chunks with a validated reconciliation or cleanup process.

Upserting new chunks does not imply deletion of old IDs that are absent from the new revision. Maintain source-to-chunk lineage and reconcile the indexed set when a source changes or disappears. Use a cleanup mode appropriate to a complete source refresh versus a partial feed, and test deletion behavior before production use.

Why the alternatives fail

B. Fresh IDs create more duplicates without removing superseded records.

C. Chunk count is not a reliable identity or lifecycle mechanism.

D. A prompt cannot reliably filter stale content when the index lacks a trustworthy revision distinction.

Design takeaway: Index maintenance needs deletion semantics as well as insertion and update semantics.

Official references: [13]

14. Cost-aware abstention

Answer B: B, with loss 71 versus A's 110; verify this trade-off on held-out and important slices.

A's loss is 10x10+10=110. B's is 4x10+31=71. Under the specified costs, fewer wrong answers outweigh the greater abstention count. This is an empirical selection on this sample, not proof that B generalizes. Validate representative coverage, slice behavior, and uncertainty before changing the release decision.

Why the alternatives fail

A. Answer count alone does not optimize the supplied asymmetric loss.

C. The question explicitly assigns ten times more cost to a wrong answer.

D. Equal sample size enables comparison but does not imply equal outcomes.

Design takeaway: Evaluate abstention with the product's error costs, not answer rate alone.

Official references: [14]

15. A release gate that aggregate accuracy misses

Answer D: Block promotion under the current gate and investigate the four critical regressions despite higher aggregate accuracy.

The candidate improves total accuracy from 910/1,000 to 946/1,000 but drops the critical slice from 10/10 to 6/10. That violates the explicit release rule. Inspect the paired failures and expand coverage where needed; a small critical sample limits broader statistical claims, but it does not authorize ignoring observed violations of the gate.

Why the alternatives fail

A. An oracle union of successes is not the performance of the candidate system.

B. An aggregate gain does not override a hard, predeclared critical-case constraint.

C. Removing inconvenient examples changes the evaluation rather than fixing the regression.

Design takeaway: Use paired comparisons and risk-relevant slices, not only one headline average.

Official references: [15]

16. Calibrating an LLM judge

Answer C: Blind system identities, counterbalance answer order, and calibrate the rubric against human-labeled examples.

The observation suggests positional bias in the evaluator. Counterbalancing or randomizing order and hiding system identities reduce avoidable cues; calibration against human judgments checks whether the rubric and judge measure the intended quality. Report residual disagreement and separate evaluator reliability from generator performance.

Why the alternatives fail

A. Generator randomness does not repair positional bias in the evaluator.

B. That institutionalizes the bias and makes candidate comparisons misleading.

D. Discarding calibration evidence prevents assessment of whether the judge is valid for the task.

Design takeaway: An evaluator is a component that must be tested, not an unquestionable oracle.

Official references: [14]

17. Localizing a RAG quality failure

Answer C: Audit retrieval and context assembly first, while checking oracle-test comparability and residual generator failures.

The controlled context substitution points to evidence supply as a major contributor. Inspect indexing coverage, retrieval recall, filtering, ranking, and context formatting on the paired failures. The result does not prove retrieval is the only issue: the oracle context may differ in useful ways, and the generator still fails some cases even with verified evidence.

Why the alternatives fail

A. The large difference under a context intervention is relevant diagnostic evidence.

B. The 95% result itself includes failures and does not prove universal correctness.

D. Longer answers do not address missing or incorrectly assembled supporting passages.

Design takeaway: Use controlled component substitutions to guide debugging without overclaiming causality.

Official references: [16]

18. Testing a provider replacement

Answer D: Run contract tests for those capabilities and representative end-to-end quality, latency, and failure cases.

A shared interface standardizes access, not every provider's capability or behavior. Test the precise tool schemas, response constraints, streaming events, usage handling, and errors the application relies on, then evaluate task quality and operational trade-offs. Adapter compatibility is necessary but not sufficient for a safe substitution.

Why the alternatives fail

A. Dropping relevant tests transfers known integration risk to users without resolving it.

B. Method compatibility does not establish semantic equivalence or feature parity.

C. Price alone ignores the behaviors required by the product contract.

Design takeaway: Treat provider migration as a contract and quality change, not just a configuration edit.

Official references: [17], [7]

19. Compression that changes the claim

Answer B: Preserve qualifications and negation in compression, retain source spans, and test entailment against the original.

The evidence is corrupted after retrieval, so the compression stage needs fidelity checks. Prefer extraction or compression that preserves conditions, exclusions, and scope; retain original spans for verification and debugging. Evaluate whether compressed statements are entailed by the source. A valid citation ID cannot rescue text whose meaning was reversed.

Why the alternatives fail

A. More retrieved candidates do not correct a transform that reverses an existing passage.

C. Brevity does not imply semantic fidelity.

D. Provenance identifies a source but does not prove that the stated claim follows from it.

Design takeaway: Measure context compression for meaning preservation, not just token reduction.

Official references: [1], [16]

20. Separating tool context from application artifacts

Answer A: Put the model-relevant summary in content and the chart payload in artifact.

ToolMessage.content is the model-facing result; artifact can hold supplementary application data not sent to the model as that message's content. This separates reasoning context from rendering data and avoids unnecessary token load. Artifacts still need appropriate handling in storage, tracing, and UI access; they are not automatically secret merely because they are excluded from model content.

Why the alternatives fail

B. A comment does not remove data from the model-facing message.

C. Artifact data is not automatically substituted for content in the model conversation.

D. Recreating exact source data from a generated summary is lossy and unnecessary.

Design takeaway: Keep model context intentional while preserving rich tool results for the application.

Official references: [18]

LangChain-M-H-Set2

Official sources | Documentation checked October 28, 2026

These original scenarios apply documented APIs and engineering reasoning to the stated constraints. References support the concepts; they do not prescribe every architectural conclusion. Standard imports and setup are omitted from illustrative snippets.

[1] Reranker integration
https://docs.langchain.com/oss/python/integrations/retrievers/cohere-reranker

[2] Guardrails and enforcement points
https://docs.langchain.com/oss/python/langchain/guardrails

[3] Tool schemas and execution context
https://docs.langchain.com/oss/python/langchain/tools

[4] Human review of tool calls
https://docs.langchain.com/oss/python/langchain/human-in-the-loop

[5] RunnableWithFallbacks
https://reference.langchain.com/python/langchain-core/runnables/fallbacks/RunnableWithFallbacks

[6] Custom middleware
https://docs.langchain.com/oss/python/langchain/middleware/custom

[7] Chat model interface
https://docs.langchain.com/oss/python/langchain/models

[8] RunnableParallel
https://reference.langchain.com/python/langchain-core/runnables/base/RunnableParallel

[9] Agent structured output
https://docs.langchain.com/oss/python/langchain/structured-output

[10] Weighted reciprocal rank fusion
https://github.com/langchain-ai/langchain/blob/master/libs/langchain/langchain_classic/retrievers/ensemble.py

[11] Runnable composition
https://reference.langchain.com/python/langchain-core/runnables/base/Runnable

[12] Document content and metadata
https://reference.langchain.com/python/langchain-core/documents/base/Document

LangChain-M-H-Set2

Official sources | Documentation checked October 28, 2026

These original scenarios apply documented APIs and engineering reasoning to the stated constraints. References support the concepts; they do not prescribe every architectural conclusion. Standard imports and setup are omitted from illustrative snippets.

[13] Source updates and indexing cleanup
https://www.langchain.com/blog/syncing-data-sources-to-vector-stores

[14] Evaluation approaches
https://docs.langchain.com/langsmith/evaluation-types

[15] Evaluation concepts and datasets
https://docs.langchain.com/langsmith/evaluation-concepts

[16] RAG evaluation tutorial
https://docs.langchain.com/langsmith/evaluate-rag-tutorial

[17] Testing LangChain agents
https://docs.langchain.com/oss/python/langchain/test

[18] ToolMessage content and artifacts
https://docs.langchain.com/oss/python/langchain/messages